
Introduction to Machine Learning in Python

Réalisé par : LOUATI Malek

EL HDOUR Hafssa

Année universitaire : 2025–2026

Date : 12 juin 2026

Table des matières

1	Introduction	2
2	Présentation du dataset	2
2.1	Distribution des classes	2
2.2	Annotations XML et localisation du chien	3
3	Analyse de l'équilibre des classes	3
4	Prétraitement des images	4
5	Conversion en matrice de données	4
6	Extraction de caractéristiques avec la PCA	5
7	Extraction de caractéristiques avec HOG	6
8	Séparation train/test et pipeline	7
8.1	Validation du découpage	7
8.2	Construction du pipeline	8
8.3	Diagnostic de généralisation et effet de la standardisation	8
9	Optimisation et résultats du modèle	9
9.1	Analyse de la matrice de confusion	10
9.2	Précision, rappel et score F1	10
10	Comparaison OvO / OvR	11
10.1	Analyse de la complexité	11
11	Conclusion	12

1 Introduction

Lors de ce projet, nous nous intéressons à la classification automatique de races de chiens, qui consiste à attribuer une catégorie à une image à partir de ses caractéristiques visuelles. L'objectif est de construire un modèle capable d'identifier la race d'un chien à partir d'une image parmi plusieurs classes. Ce problème relève de l'apprentissage supervisé puisque chaque image du jeu de données est associée à une étiquette indiquant sa race.

Cette tâche présente plusieurs difficultés. Les races peuvent avoir des caractéristiques visuelles proches, les conditions d'éclairage varient d'une image à l'autre et les arrière-plans sont souvent différents. De plus, les images ne possèdent pas toutes la même taille, ce qui nécessite un prétraitement préalable.

Pour répondre à cette problématique, nous mettons en œuvre une chaîne complète de traitement comprenant le prétraitement des images, l'extraction de caractéristiques à l'aide de la PCA et du descripteur HOG, puis l'entraînement d'un classificateur SVM. Les performances du modèle sont ensuite évaluées à l'aide de différentes métriques afin de mesurer sa capacité à reconnaître correctement les races de chiens.

2 Présentation du dataset

Le projet repose sur la base de données *SmallDB*, une version réduite du jeu de données *Stanford Dogs Dataset*. Cette base contient des images de différentes races de chiens ainsi que leurs annotations XML associées.

Chaque image est associée à une étiquette correspondant à sa race. Les fichiers XML permettent quant à eux de localiser précisément le chien dans l'image grâce à une boîte englobante (*bounding box*).

2.1 Distribution des classes

La Figure 1 présente la répartition du nombre d'images pour chacune des races présentes dans le dataset.

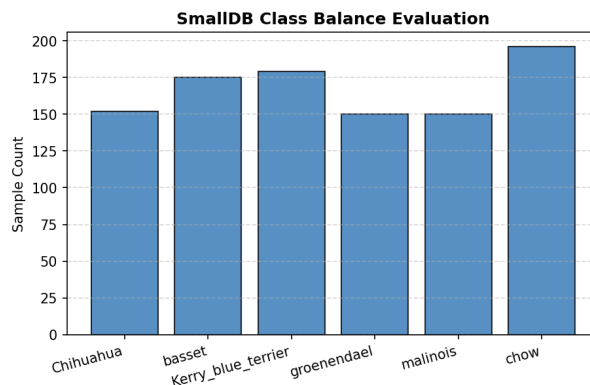


FIGURE 1 – Répartition des images par race dans le dataset SmallDB.

On observe que les différentes classes sont relativement équilibrées. Le nombre d'images varie approximativement entre 150 et 200 observations par race. Cette homogénéité constitue un avantage pour l'apprentissage, car elle réduit le risque qu'un modèle privilégie une

classe particulière simplement parce qu'elle est surreprésentée. On peut donc s'attendre à un apprentissage plus équitable entre les différentes races.

2.2 Annotations XML et localisation du chien

Les annotations XML fournies avec le dataset permettent d'identifier précisément la position du chien dans chaque image. Ces informations sont utilisées pour construire une boîte englobante (*bounding box*) autour de l'animal.

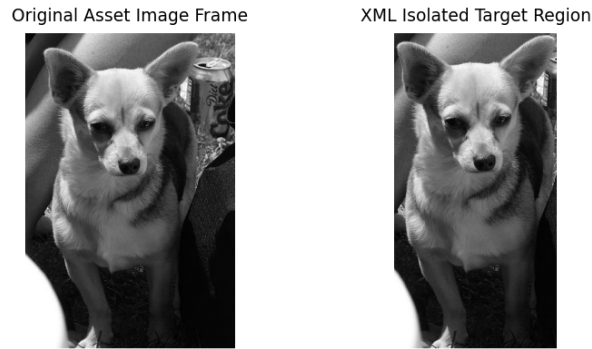


FIGURE 2 – Extraction du chien à partir des annotations XML.

La Figure 2 illustre ce mécanisme. L'image originale contient à la fois le chien et son environnement. Grâce aux coordonnées extraites du fichier XML, il est possible d'isoler uniquement la région contenant l'animal. Cette opération permet de concentrer l'apprentissage sur l'objet d'intérêt et de limiter l'influence du fond de l'image, qui pourrait introduire du bruit dans les données.

3 Analyse de l'équilibre des classes

Avant d'entraîner un modèle, il est important de vérifier que les classes sont relativement équilibrées. En effet, si une race est beaucoup plus représentée que les autres, le modèle peut obtenir une bonne accuracy simplement en prédisant toujours cette classe majoritaire.

Pour mesurer cet équilibre, nous utilisons l'entropie de Shannon. Elle est définie par :

$$H(P) = - \sum_{i=1}^K p_i \log(p_i)$$

où K représente le nombre de classes et p_i la proportion d'images appartenant à la classe i .

Une entropie faible indique un dataset déséquilibré, tandis qu'une entropie élevée indique une distribution plus homogène des classes. Dans le code, cette mesure est calculée à l'aide des fonctions `entropy()` et `entropy_breeds()`.

La valeur obtenue dans notre cas est :

$$H = 1.7863$$

Cette valeur est très proche de l'entropie maximale théorique ($\log(6) \approx 1.79$), ce qui indique une répartition presque uniforme des races dans le dataset.

Dans un classificateur basé sur les distances comme le k-NN, un fort déséquilibre favoriserait la classe majoritaire lors du vote des voisins les plus proches. Dans notre cas, l'entropie élevée montre que cet effet est négligeable. Les performances du modèle sont donc davantage limitées par les similitudes visuelles entre certaines races que par un problème d'équilibre des classes.

4 Prétraitement des images

Les images du dataset possèdent des tailles différentes, alors que les modèles de machine learning nécessitent des données de dimension fixe. Une étape de prétraitement est donc nécessaire.

La fonction `read_and_crop()` utilise les annotations XML pour extraire automatiquement la zone contenant le chien. Les images sont ensuite redimensionnées en 64×64 pixels.

Afin d'éviter les déformations géométriques, la fonction `resize_and_pad()` conserve les proportions de l'image et ajoute du padding pour obtenir une image carrée.

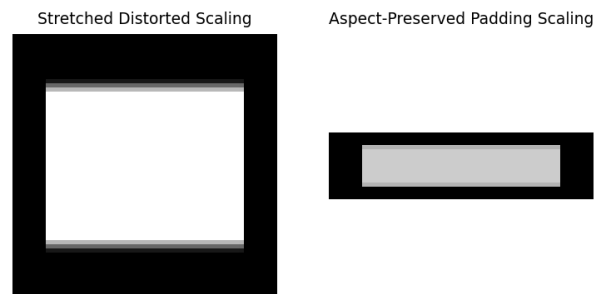


FIGURE 3 – Comparaison entre un redimensionnement déformant et un redimensionnement avec conservation des proportions grâce au padding.

La Figure 3 montre qu'un redimensionnement direct déforme les proportions du chien et modifie les distances entre pixels. Deux images similaires peuvent alors apparaître artificiellement différentes, ce qui perturbe les calculs de distance utilisés par les algorithmes de classification.

Le padding évite ce problème en conservant le rapport hauteur/largeur de l'image. Les caractéristiques extraites par la PCA et HOG restent ainsi cohérentes avec la géométrie réelle du chien.

5 Conversion en matrice de données

Les algorithmes de machine learning ne peuvent pas traiter directement des images. Chaque image doit donc être transformée en un vecteur numérique.

Après le prétraitement, toutes les images sont redimensionnées à 64×64 pixels. La fonction `convert_ndarrays2data_matrix()` convertit alors chaque image en un vecteur de 4096 valeurs.

La matrice finale contient une ligne par image et une colonne par pixel. Cette représentation permet d'utiliser directement les outils de classification de `scikit-learn`.

Cependant, cette approche génère des données de grande dimension et ne conserve pas les informations de couleur. Une étape d'extraction de caractéristiques est donc nécessaire avant la classification.

6 Extraction de caractéristiques avec la PCA

L'Analyse en Composantes Principales (PCA) est une méthode de réduction de dimension qui permet de représenter les données dans un espace de plus faible dimension tout en conservant l'essentiel de l'information.

L'objectif est de réduire le nombre de variables, de diminuer le bruit présent dans les données et de faciliter l'apprentissage des modèles de classification.



FIGURE 4 – Reconstruction d'images à l'aide de différentes composantes principales.

La Figure 4 illustre l'effet du nombre de composantes sur la qualité de reconstruction. Lorsque le nombre de composantes augmente, les détails de l'image sont progressivement restaurés et la représentation devient plus fidèle à l'image originale.

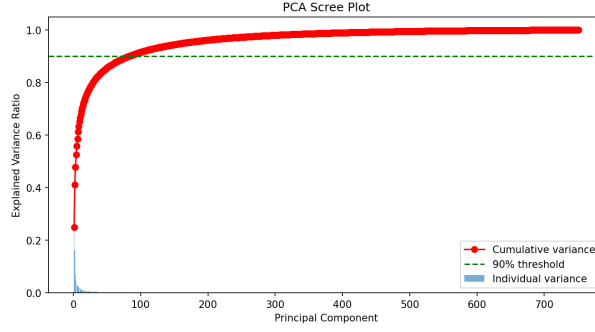


FIGURE 5 – Variance expliquée par les composantes principales.

La Figure 5 montre l'évolution de la variance expliquée en fonction du nombre de composantes principales. On observe qu'environ 90% de la variance totale est capturée avec une fraction limitée des composantes, ce qui justifie l'utilisation de la PCA pour réduire la dimension des données.

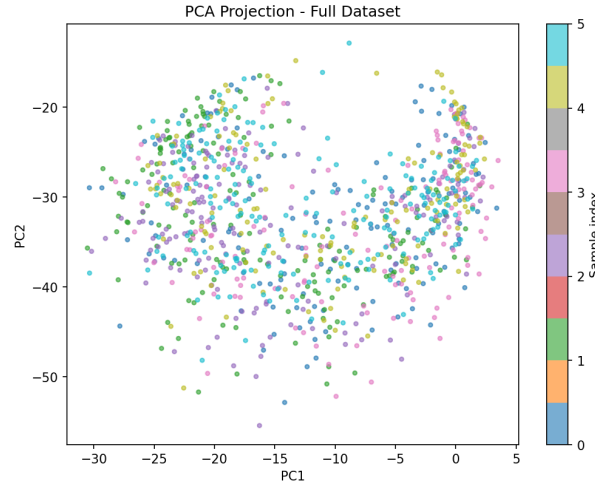


FIGURE 6 – Projection des données sur les premières composantes principales.

La Figure 6 représente les images projetées sur les deux premières composantes principales. Cette visualisation permet d'observer la structure globale du dataset. Certaines races apparaissent partiellement regroupées, tandis que d'autres présentent un recouvrement important, ce qui traduit une difficulté intrinsèque de séparation.

7 Extraction de caractéristiques avec HOG

Le descripteur HOG (*Histogram of Oriented Gradients*) est une méthode d'extraction de caractéristiques basée sur les contours et les variations locales d'intensité. Contrairement à la PCA qui capture l'information globale, HOG se concentre sur la forme et la structure des objets présents dans l'image.

La première étape consiste à calculer les gradients horizontaux et verticaux à l'aide des opérateurs de Sobel. Ces gradients permettent d'estimer les variations d'intensité et les orientations des contours.

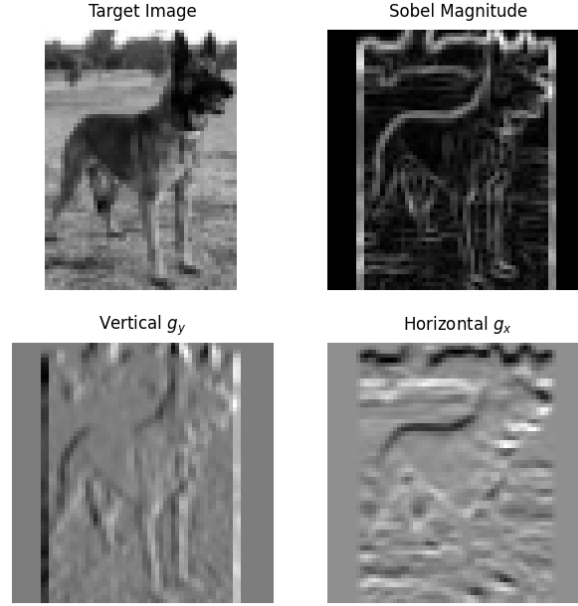


FIGURE 7 – Extraction des contours à l'aide des opérateurs de Sobel.

La Figure 7 montre les caractéristiques extraites à partir des opérateurs de Sobel. Les contours du chien ainsi que les principales transitions d'intensité sont clairement mis en évidence.

Ces informations servent ensuite à construire les descripteurs HOG sous forme d'histogrammes d'orientations. Les caractéristiques obtenues capturent efficacement la silhouette et la structure de l'animal, ce qui constitue une information particulièrement utile pour la classification des races de chiens.

L'utilisation conjointe de la PCA et de HOG permet ainsi de combiner une représentation globale des images avec une description locale basée sur les contours.

8 Séparation train/test et pipeline

Avant l'apprentissage, le dataset a été divisé en deux parties : un ensemble d'entraînement et un ensemble de test. Le paramètre `stratify=labels` a été utilisé afin de conserver une proportion similaire de races dans les deux ensembles.

8.1 Validation du découpage

Pour valider le découpage train/test, les distributions de classes ont été comparées avec une similarité cosinus :

$$\text{Similarity} = \frac{\mathbf{p}_{train} \cdot \mathbf{p}_{test}}{\|\mathbf{p}_{train}\| \|\mathbf{p}_{test}\|} \quad (1)$$

Le score obtenu est :

$$\text{Similarity} = 0.999951 \quad (2)$$

Cette valeur proche de 1 montre que les ensembles d'entraînement et de test conservent presque la même distribution de classes. Si cette similarité était faible, le modèle serait

entraîné sur une distribution différente de celle du test set, ce qui rendrait l'accuracy moins fiable.

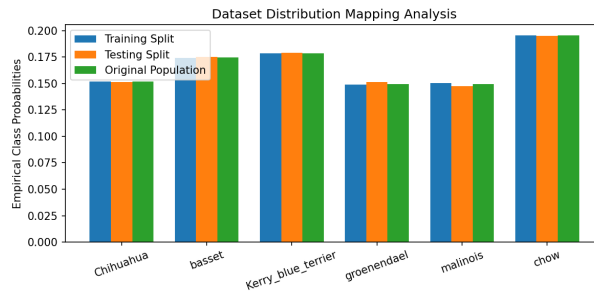


FIGURE 8 – Distribution des classes dans le dataset original, le train set et le test set.

8.2 Construction du pipeline

Un pipeline *scikit-learn* a été utilisé pour regrouper les étapes du modèle : normalisation, extraction de caractéristiques PCA/HOG, standardisation et classification par SVM.

```
all_features = FeatureUnion([
    ("pca_info", PCAInfoPreprocessing()),
    ("edge_info", EdgeInfoPreprocessing())
])

pipeline_svc = Pipeline([
    ("minmax_scaler", MinMaxScaler()),
    ("all_features", all_features),
    ("standard_scaler", StandardScaler()),
    ("svc", SVC(kernel="linear"))
])
```

`FeatureUnion` combine les caractéristiques extraites par PCA et HOG dans un seul vecteur de features. Le pipeline permet aussi de séparer clairement `fit` et `transform` : les transformations sont apprises uniquement sur les données d'entraînement, puis appliquées aux données de validation ou de test. Cela limite les risques de fuite de données.

8.3 Diagnostic de généralisation et effet de la standardisation

Afin d'évaluer l'influence de la standardisation des caractéristiques, le pipeline a été exécuté deux fois : une première fois avec le `StandardScaler` et une seconde fois sans cette étape.

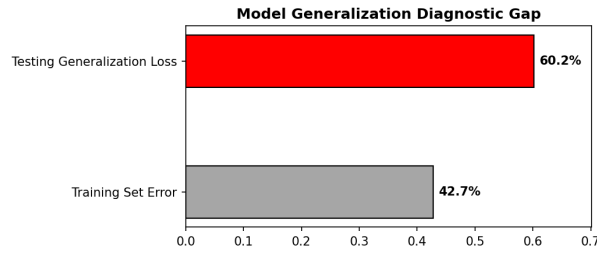


FIGURE 9 – Erreur d’entraînement et erreur de test sans standardisation.

Sans standardisation, l’erreur d’entraînement est d’environ 42.7% tandis que l’erreur de test atteint 60.2%. L’écart observé montre que le modèle rencontre des difficultés à généraliser correctement sur les données de test.

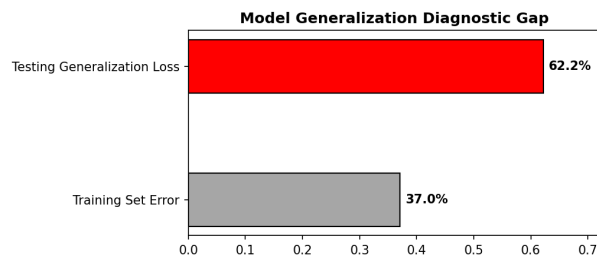


FIGURE 10 – Erreur d’entraînement et erreur de test avec standardisation.

Après standardisation, l’erreur d’entraînement diminue à environ 37.0%, tandis que l’erreur de test reste proche de 62.2%. La standardisation améliore donc l’apprentissage, mais son effet sur la généralisation reste limité.

La comparaison des Figures 9 et 10 montre que le scaler influence le comportement du classificateur. Théoriquement, sans standardisation, les composantes PCA peuvent dominer le calcul des distances du k-NN en raison de leur variance plus élevée. Cependant, dans notre cas, les performances semblent davantage limitées par les similitudes visuelles entre certaines races que par cet effet.

9 Optimisation et résultats du modèle

Après la construction du pipeline, les hyperparamètres ont été optimisés avec `GridSearchCV`. Contrairement à l’entraînement classique, qui ajuste les paramètres internes du modèle, `GridSearchCV` teste plusieurs configurations externes comme le noyau SVM, C et γ .

Après plusieurs essais, les paramètres HOG ont été retirés de la grille, car leur suppression a amélioré le score moyen de validation croisée. `GridSearchCV` choisit les meilleurs hyperparamètres en utilisant seulement l’ensemble d’entraînement. L’ensemble de test est gardé pour l’évaluation finale.

La meilleure configuration obtenue est présentée dans le tableau suivant :

Kernel	C	Gamma	CV Score
RBF	20	0.005	57.52%

TABLE 1 – Meilleure configuration obtenue avec `GridSearchCV`.

Le modèle final utilise donc un noyau RBF avec $C = 20$ et $\gamma = 0.005$. Sur l'ensemble de test, il obtient une accuracy de 54.98%.

9.1 Analyse de la matrice de confusion

Une matrice de confusion a été utilisée pour analyser les erreurs du modèle sur l'ensemble de test. Les valeurs sur la diagonale indiquent les prédictions correctes, tandis que les valeurs hors diagonale montrent les confusions entre les races.

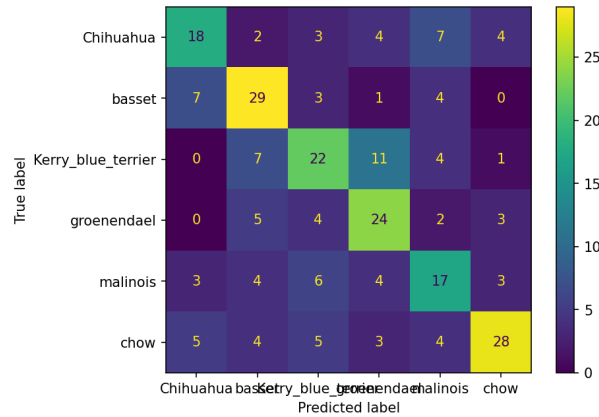


FIGURE 11 – Matrice de confusion obtenue sur l'ensemble de test.

La matrice montre que certaines classes sont mieux reconnues que d'autres. Les valeurs élevées sur la diagonale correspondent aux races les mieux prédites, tandis que les valeurs hors diagonale indiquent les races les plus souvent confondues.

9.2 Précision, rappel et score F1

Les métriques precision, recall et F1-score sont calculées à partir de la matrice de confusion. Pour chaque classe, les valeurs de la matrice permettent d'identifier les vrais positifs, les faux positifs et les faux négatifs.

$$Precision = \frac{TP}{TP + FP} \quad (3)$$

$$Recall = \frac{TP}{TP + FN} \quad (4)$$

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (5)$$

Le modèle final obtient une accuracy de 54.98% sur l'ensemble de test. Le tableau suivant présente les métriques détaillées par classe.

Ces métriques complètent l'accuracy globale, car elles permettent d'analyser les performances classe par classe. Les résultats montrent que la classe *chow* obtient la meilleure précision avec 0.72, ce qui signifie que les prédictions de cette classe sont relativement fiables. Cependant, son rappel est de 0.57, ce qui indique que plusieurs images de cette classe sont encore confondues avec d'autres races. La classe *malinois* présente les résultats les plus faibles, avec un F1-score de 0.45. Globalement, l'accuracy de 54.98% montre que le modèle apprend certaines structures discriminantes, mais que la séparation entre plusieurs races reste difficile.

Classe	Precision	Recall	F1-score	Support
Chihuahua	0.55	0.47	0.51	38
Basset	0.57	0.66	0.61	44
Kerry blue terrier	0.51	0.49	0.50	45
Groenendael	0.51	0.63	0.56	38
Malinois	0.45	0.46	0.45	37
Chow	0.72	0.57	0.64	49
Accuracy	—	—	0.55	251
Macro avg	0.55	0.55	0.55	251
Weighted avg	0.56	0.55	0.55	251

TABLE 2 – Rapport de classification du modèle final sur l’ensemble de test.

10 Comparaison OvO / OvR

Pour adapter le SVM à un problème multiclass, deux stratégies ont été comparées : One-vs-One (OvO) et One-vs-Rest (OvR). OvO entraîne un modèle pour chaque paire de classes, tandis que OvR entraîne un modèle par classe.

Stratégie	Accuracy	Temps	Modèles
OvO	55.38%	2.22 s	15
OvR	55.78%	2.19 s	6

TABLE 3 – Comparaison expérimentale entre OvO et OvR.

Les deux stratégies donnent des résultats similaires. OvR est légèrement meilleur en accuracy, en temps d’entraînement et en nombre de modèles.

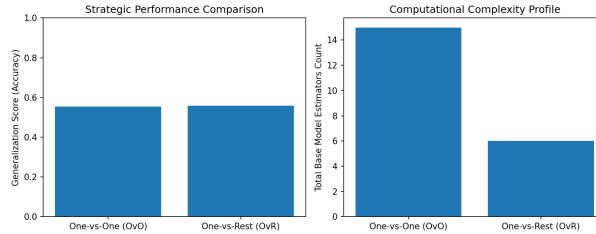


FIGURE 12 – Comparaison des performances et de la complexité des stratégies OvO et OvR.

10.1 Analyse de la complexité

Pour la stratégie One-vs-Rest, un modèle est entraîné par classe :

$$N_{OvR} = C \quad (6)$$

Pour la stratégie One-vs-One, un modèle est entraîné pour chaque paire de classes :

$$N_{OvO} = \binom{C}{2} = \frac{C(C-1)}{2} \quad (7)$$

Lorsque C devient très grand :

$$N_{OvO} = \frac{C^2 - C}{2} \sim \frac{C^2}{2} \quad (8)$$

Donc OvO a une complexité quadratique :

$$N_{OvO} = O(C^2) \quad (9)$$

alors que OvR a une complexité linéaire :

$$N_{OvR} = O(C) \quad (10)$$

Dans notre dataset, il y a six classes :

$$N_{OvO} = 15 \quad N_{OvR} = 6 \quad (11)$$

Pour $C = 1000$ classes :

$$N_{OvO} = \frac{1000 \times 999}{2} = 499\,500 \quad (12)$$

alors que :

$$N_{OvR} = 1000 \quad (13)$$

Ainsi, OvO devient difficilement applicable à grande échelle, tandis que OvR reste plus scalable.

11 Conclusion

Ce projet a permis de construire une chaîne complète de classification d'images de chiens avec des méthodes classiques de machine learning.

Les images ont été prétraitées, redimensionnées et converties en matrices de données. Ensuite, des caractéristiques ont été extraites avec la PCA et les descripteurs HOG. Ces caractéristiques ont été utilisées pour entraîner un classificateur SVM.

L'optimisation avec `GridSearchCV` a permis d'identifier le noyau RBF comme la meilleure configuration, avec $C = 20$ et $\gamma = 0.005$. Le modèle final obtient une accuracy de 54.98% sur l'ensemble de test.

La matrice de confusion montre que certaines races sont correctement reconnues, tandis que d'autres sont plus facilement confondues. L'analyse de la précision, du rappel et du score F1 permet de mieux comprendre ces erreurs.

Enfin, la comparaison entre OvO et OvR montre que les deux stratégies ont des performances proches sur notre dataset. Cependant, l'analyse de complexité montre que OvR est plus scalable lorsque le nombre de classes devient très élevé.

Comme perspectives d'amélioration, il serait possible d'utiliser davantage de données, d'appliquer de l'augmentation de données ou d'utiliser des modèles plus avancés comme les réseaux de neurones convolutifs ou le transfert d'apprentissage.